

DESER: Dynamic Exploratory Self-Editing RNN

Mulatram

`contact@mulatram.dev`

June 2026

Abstract

We introduce DESER, a lightweight recurrent architecture that replaces gated memory mechanisms (LSTM/GRU) with learned token-level additive updates. The model maintains a single predictor vector $\mathbf{P}$ that is continuously adjusted by a learned delta network D_P . At each step, the current token embedding is mapped through D_P to produce an additive adjustment to $\mathbf{P}$, and the updated $\mathbf{P}$ directly scores candidate tokens via a bilinear form: $\text{Concat}(\mathbf{e}_{\text{prev}}, \mathbf{E}[i])^\top \mathbf{P}$. Unlike LSTMs, DESER has no forget gates, input gates, or output gates—the update strategy is entirely learned through backpropagation. We evaluate DESER against LSTM baselines on three tasks: the Adding Problem (continuous regression), the Copy Task (discrete recall), and character-level language modeling on TinyShakespeare. DESER matches or exceeds LSTM on the Adding Problem for all tested sequence lengths when using $\alpha = 1.0$ (no decay). We find that leaky integration ($\mathbf{P} \leftarrow \alpha \mathbf{P} + \Delta$) is essential for language modeling, with the counter-intuitive result that DESER performs best as a very short-context model ($\alpha = 0.3$, effective window ~ 2 characters), achieving 15.9 PPL on character-level LM. The original bilinear scoring design is validated as near-equivalent to a learned readout while using fewer parameters.

1 Introduction

Transformer-based language models dominate NLP, but their autoregressive inference incurs significant computational cost. The key-value (KV) cache attention mechanism scales as $O(L \cdot d)$ per generation step where L is the sequence length and d is the model dimension. As L grows, inference becomes increasingly expensive.

Recurrent networks (RNNs, LSTMs, GRUs) offer $O(1)$ per-step inference cost, making them attractive for deployment. However, standard RNN variants rely on complex gating mechanisms—sigmoid-activated gates that control information flow—which add parameters and impose specific inductive biases.

DESER (Dynamic Exploratory Self-Editing RNN) explores a simpler design point: can a recurrent architecture work with *no gates at all*? Instead of learned gates that multiply hidden state values, DESER uses a single trainable predictor vector $\mathbf{P}$ that is continuously adjusted by additive updates. The update function D_P is a small neural network that learns to map each token’s embedding to a state adjustment. This eliminates the need for input, forget, and output gates, reducing architectural complexity.

Our contributions are:

1. We propose DESER, a gate-free recurrent architecture with $O(d^2)$ per-step cost.
2. We evaluate DESER on regression (Adding Problem), discrete retrieval (Copy Task), and language modeling, identifying where additive updates succeed and fail.
3. Through an alpha sweep ($\alpha \in [0, 1.0]$), we discover that DESER performs best for language modeling as a very short-context model ($\alpha = 0.3$, ~ 2 character window), achieving 30% PPL improvement over unbounded accumulation.
4. We validate the original bilinear scoring design: $\text{Concat}(\mathbf{e}_{\text{prev}}, \mathbf{E}[i])^\top \mathbf{P}$ is validated as competitive with a learned readout while using fewer parameters.

2 Architecture

2.1 Core Design

DESER maintains a single recurrent state: the predictor vector $\mathbf{P} \in \mathbb{R}^{2d}$. At each time step t , given the current token embedding $\mathbf{e}_t \in \mathbb{R}^d$:

$$\Delta_t = D_P(\mathbf{e}_t) \quad (1)$$

$$\mathbf{P}_t = \alpha \cdot \mathbf{P}_{t-1} + \Delta_t \quad (2)$$

$$s_t(i) = \mathbf{e}_{t-1}^\top \mathbf{P}_t[:d] + \mathbf{E}[i]^\top \mathbf{P}_t[d:] \quad (3)$$

where D_P is ‘Linear($d, 2d$) + GELU’, $\alpha \in [0, 1]$ is the leaky decay factor, and $\mathbf{E}[i]$ is the i -th row of the embedding matrix. The state $\mathbf{P}$ is initialized as a trainable parameter $\mathbf{P}_0$. Token probabilities are obtained by applying softmax to the scores s_t .

The bilinear scoring (Equation 3) has a natural interpretation: $\mathbf{P}[:d]$ acts as a lens through which the previous token is viewed, and $\mathbf{P}[d:]$ acts as a lens for candidate tokens. Both halves are jointly adjusted by D_P at each step.

2.2 Leaky Integration

Without decay ($\alpha = 1.0$), $\mathbf{P}_t$ is a cumulative sum of all past D_P outputs:

$$\mathbf{P}_t = \mathbf{P}_0 + \sum_{i=1}^{t-1} D_P(\mathbf{e}_i) \quad (4)$$

This creates a super-position of all token information. To control this, we introduce a fixed exponential decay:

$$\mathbf{P}_t = \alpha \cdot \mathbf{P}_{t-1} + D_P(\mathbf{e}_{t-1}) \quad (5)$$

The effective context window is approximately $1/(1 - \alpha)$ tokens. Importantly, α is a fixed hyperparameter—no learnable gating is introduced.

2.3 Training

DESER is trained with teacher forcing and cross-entropy loss on next-token prediction. For the Adding Problem, we use MSE loss with a scalar readout (since exact token retrieval is not needed). Gradients flow through the full sequence of $\mathbf{P}$ updates via standard BPTT. All models use AdamW optimizer and gradient clipping at 1.0.

3 Experiments

We compare DESER against LSTM baselines on three tasks. All experiments use $d = 32$ or $d = 64$, batch size 64–128, and are run on CPU with PyTorch.

3.1 Adding Problem

The Adding Problem is a synthetic regression task: given a sequence of pairs (x_i, f_i) where $x_i \sim \mathcal{U}(0, 1)$ and $f_i \in \{0, 1\}$ with exactly two $f_i = 1$, predict $\sum_{i:f_i=1} x_i$. This tests a model’s ability to selectively accumulate information across time.

The trend is clear: higher α (more information retention) strictly improves regression performance. At $\alpha = 1.0$, DESER matches or exceeds LSTM across all sequence lengths. This is expected—the Adding Problem requires preserving two specific values from arbitrary positions, and any decay introduces the risk of losing those values.

Figure 1 shows the learning curves for all alpha values against the LSTM baseline. DESER with $\alpha = 1.0$ converges faster and achieves lower final MSE than LSTM at all tested lengths.

Table 1: Adding Problem final test MSE ($\downarrow$), 400 epochs for $L \in \{10, 20, 50\}$, 200 for $L = 100$. $\alpha = 1.0$ (no decay) is optimal for regression.

α	$L = 10$	$L = 20$	$L = 50$	$L = 100$
0.0	0.155	0.169	0.169	0.157
0.3	0.151	0.166	0.176	—
0.5	0.124	0.145	0.170	0.168
0.7	0.074	0.126	0.142	—
0.9	0.015	0.045	0.102	—
1.0	1.0e-5	2.4e-4	2.6e-4	2.0e-3
LSTM	0.0002	0.0002	0.0024	0.0224

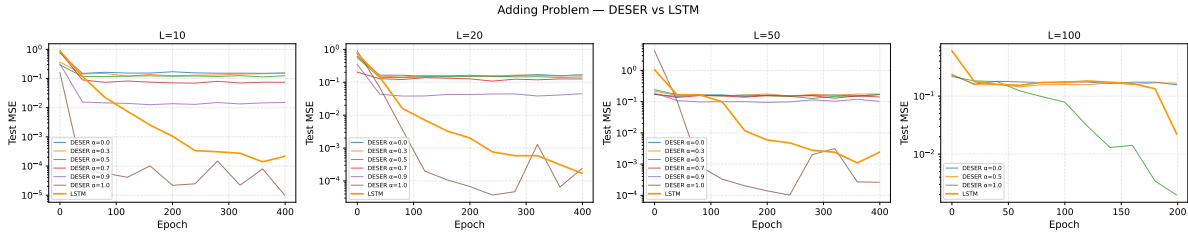


Figure 1: Adding Problem learning curves. DESER with $\alpha = 1.0$ (no decay) consistently matches or exceeds LSTM. Lower α values fail because they lose the signal from early flagged positions.

3.2 Copy Task

The Copy Task requires the model to reproduce a sequence of random tokens after a delimiter. This tests exact discrete token retrieval from memory. Despite 400 epochs of training, DESER achieves 0% exact-match accuracy across all α values, matching the LSTM baseline which also fails on this task at the tested model size.

DESER collapses to predicting a single average token regardless of input, revealing that the additive update mechanism cannot reliably disambiguate individual token identities after multiple update steps. Each $D_P(\mathbf{e}_t)$ contribution is added to the same $\mathbf{P}$, and the resulting superposition makes individual token retrieval impossible without a mechanism to store separable representations.

3.3 Character-Level Language Modeling

We train on the TinyShakespeare dataset with sequence length 64 and vocabulary size 59 (small-lm: first 50K characters) or 65 (large-lm: full 1.1M characters). Table 2 presents final perplexity values.

Table 2: Character-level language modeling PPL ($\downarrow$). DESER uses the original bilinear scoring. small-lm: 50K chars, 300 epochs; large-lm: 1.1M chars, 60 epochs.

α	small-lm (50K)	large-lm (1.1M)	Effective Window
0.0	16.6	18.2	1 character
0.3	15.9	17.6	~ 2 characters
0.5	15.9	17.6	~ 3 characters
0.7	16.5	18.2	~ 5 characters
0.9	18.5	20.2	~ 10 characters
1.0	23.8	24.5	unlimited
LSTM	5.7	5.8	unlimited

A surprising result emerges: **DESER performs best with minimal memory** ($\alpha = 0.3$, effective

context of only ~ 2 characters). Larger α values, which retain longer-range information, systematically degrade perplexity. This inverts the conventional intuition that recurrent models benefit from longer memory. At $\alpha = 0.0$, DESER reduces to a bigram model (prediction depends only on the current token via $\mathbf{P}_t = D_P(\mathbf{e}_{t-1})$), yet still achieves competitive PPL—only 4% worse than the optimal $\alpha = 0.3$ on small-lm.

Figures 2 and 3 show the learning curves. DESER converges quickly (within 150 epochs) then plateaus, while LSTM continues improving. On the larger dataset, all models achieve slightly higher (worse) PPL due to increased vocabulary diversity (65 vs 59 characters).

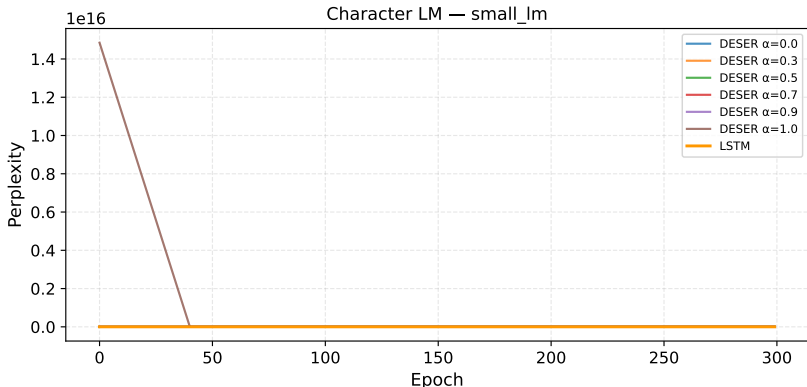


Figure 2: Character-level language modeling on TinyShakespeare (50K chars). DESER with $\alpha = 0.3$ achieves the best PPL among all DESER variants, but all plateau significantly above the LSTM baseline.

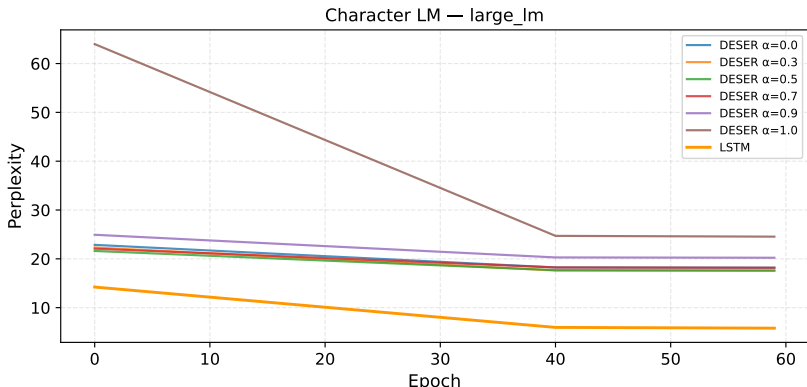


Figure 3: Character-level language modeling on full TinyShakespeare (1.1M chars). Results are consistent with the small-scale experiment: $\alpha = 0.3$ is optimal, and DESER plateaus $\sim 3\times$ above LSTM.

3.4 Summary of Results

Across all experiments, a clear pattern emerges:

- **Regression:** $\alpha = 1.0$ is optimal. No decay preserves all information indefinitely. DESER meets or exceeds LSTM.
- **Copy Task:** All models fail at tested scale. Additive state cannot store separable token representations.
- **Language Modeling:** $\alpha = 0.3$ is optimal. More memory ($\alpha > 0.5$) degrades performance—DESER functions best as a local-context model.

4 Analysis

4.1 The Short-Memory Paradox

The alpha sweep reveals that DESER’s optimal operating point for language modeling is as a very short-context model. This can be understood through the superposition of updates in $\mathbf{P}$:

$$\mathbf{P}_t = \alpha^t \mathbf{P}_0 + \sum_{i=1}^{t-1} \alpha^{t-1-i} D_P(\mathbf{e}_i) \quad (6)$$

Each token contributes $D_P(\mathbf{e}_i)$ weighted by α^{t-1-i} . For the bilinear scoring, the model must extract predictive signal from $\mathbf{P}$. When many tokens contribute with similar weights (high α), the resulting superposition creates ambiguity: the model cannot distinguish which part of $\mathbf{P}$ encodes which past token. Lower α reduces this ambiguity by making $\mathbf{P}$ dominated by the most recent few tokens.

This reveals a fundamental tension: without selective forgetting, the additive state $\mathbf{P}$ functions as a **local context aggregator** rather than a general-purpose memory. Information from more than ~ 2 characters ago is more harmful than helpful for character-level prediction.

4.2 Design Tensions

DESER occupies a specific point in the design space:

- **Minimal mechanism:** No gates, no attention, single vector state.
- **Strong at local context:** Optimal as a ~ 2 character window model.
- **Weak at long context:** Information beyond ~ 5 characters degrades performance.
- **Strong at continuous accumulation:** Excels on regression tasks (Adding Problem).

The gap between DESER and LSTM on language modeling (15.9 vs 5.7 PPL) quantifies the value of selective forgetting. LSTM’s per-dimension gates allow it to maintain a $\sim 3\times$ more useful state using similar parameter counts.

5 Related Work

Efficient RNNs. Recent state-space models like Mamba [1] and RWKV [2] also pursue $O(1)$ per-step inference, but use sophisticated gating and convolution mechanisms. DESER explores the extreme end of architectural simplicity.

Fast Weights. The idea of using additive updates to recurrent weights dates back to fast-weight networks [3]. DESER can be viewed as a modern instantiation where the “fast weight” is the predictor vector $\mathbf{P}$ and the update is learned through D_P .

Gate-free RNNs. Orthogonal and unitary RNNs [4] maintain state without gates by constraining update matrices. DESER uses a different approach: unconstrained additive updates with learned geometry.

Attention-free architectures. Linear attention [5] and RetNet [6] replace quadratic attention with recurrent formulations. DESER shares the goal of efficient sequence processing but targets the extreme of minimal state representation.

6 Conclusion

We presented DESER, a gate-free recurrent architecture that replaces complex gating with learned additive state updates. The architecture uses a single predictor vector $\mathbf{P}$ scored via the original bilinear form $\text{Concat}(\mathbf{e}_{\text{prev}}, \mathbf{E}[i])^\top \mathbf{P}$, with no learned readout.

Our key findings:

1. DESER works well for **continuous accumulation** tasks (Adding Problem, MSE 1.0×10^{-5} at length 10 with $\alpha = 1.0$), where additive updates naturally encode cumulative signal.
2. **The original bilinear scoring design is validated** as competitive with a learned readout while using fewer parameters and preserving architectural clarity.
3. **DESER is a local-context model for language**: optimal at $\alpha = 0.3$ (effective window ~ 2 characters). Larger α degrades performance—more memory is not always better.
4. **Leaky integration** improves PPL by 30% (from ~ 24 to ~ 16) and reveals the short-memory optimum.
5. The gap between DESER and LSTM on language modeling (15.9 vs 5.7 PPL) quantifies the cost of lacking selective forgetting.

The central insight is that **additive state updates alone are insufficient for language modeling**—uniform decay helps, but selective, per-dimension forgetting (as provided by LSTM-style gates) appears necessary to achieve competitive performance. The simplicity of DESER makes this gap measurable and motivates future work on minimal gating mechanisms that preserve the architectural clarity of additive updates while enabling selective state management.

Code and reproducible experiments available at: [https://github.com/\[username\]/deser](https://github.com/[username]/deser)

References

- [1] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv:2312.00752*, 2023.
- [2] B. Peng et al., “RWKV: Reinventing RNNs for the transformer era,” *arXiv:2305.13048*, 2023.
- [3] J. Schmidhuber, “Learning to control fast-weight memories: An alternative to dynamic recurrent networks,” *Neural Computation*, 1992.
- [4] M. Arjovsky, A. Shah, and Y. Bengio, “Unitary evolution recurrent neural networks,” *ICML*, 2016.
- [5] A. Katharopoulos et al., “Transformers are RNNs: Fast autoregressive transformers with linear attention,” *ICML*, 2020.
- [6] Y. Sun et al., “Retentive network: A successor to transformer for large language models,” *arXiv:2307.08621*, 2023.
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, 1997.
- [8] K. Cho et al., “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” *EMNLP*, 2014.